

Aula 37: Frontend do Chat - Teoria e Prática

Duração: 4 horas (2h teoria + 2h prática)

Parte Teórica (2h) - WebSockets no Frontend

1. Como o Frontend se Conecta ao Backend?

- **WebSocket API:** Interface nativa do navegador (`new WebSocket(url)`).
- **STOMP over WebSockets:** Usamos bibliotecas como `@stomp/stompjs` para lidar com mensagens estruturadas.
- **Fluxo de Comunicação:**
 1. Cliente se conecta ao endpoint do backend (`ws://localhost:8080/ws-chat`).
 2. Inscreve-se em tópicos (ex: `/topic/public`) para receber mensagens.
 3. Publica mensagens em destinos (ex: `/app/chat.send`).

2. Gerenciamento de Estado no Vue 3

- **Composables (Hooks Reutilizáveis):**
 - `useChat()` para encapsular lógica de WebSocket.
- **Pinia (ou Vuex):** Opcional para estado global (ex: lista de usuários online).

3. Estrutura do Componente de Chat

- **Elementos Básicos:**
 - Lista de mensagens (`<q-list>`).
 - Campo de texto (`<q-input>`) + botão de envio.
 - Indicador de "digitando..." (opcional).

Parte Prática (2h) - Implementando o Frontend

1. Configurando o STOMP no Vue (1h)

Arquivo: `src/composables/useChat.ts`

typescript

```
import { ref } from 'vue';
import { Client } from '@stomp/stompjs';

export function useChat() {
  const messages = ref<Message[]>([]);
  const stompClient = new Client({
    brokerURL: 'ws://localhost:8080/ws-chat',
    onConnect: () => {
      stompClient.subscribe('/topic/public', (message) => {
        messages.value.push(JSON.parse(message.body));
      });
    },
  });

  // Conecta ao WebSocket
  stompClient.activate();

  // Envia mensagem para o backend
  const sendMessage = (content: string, sender: string) => {
    stompClient.publish({
      destination: '/app/chat.send',
      body: JSON.stringify({ content, sender }),
    });
  };

  return { messages, sendMessage };
}
```

2. Componente do Chat (1h)

Arquivo: src/components/ChatRoom.vue

vue

```
<template>
  <div class="chat-container">
    <q-list bordered class="message-list">
      <q-item v-for="msg in messages" :key="msg.id">
        <strong>{{ msg.sender }}:</strong> {{ msg.content }}
      </q-item>
    </q-list>

    <q-input
      v-model="newMessage"
      label="Digite sua mensagem"
      @keyup.enter="handleSend"
    >

    <template #append>
      <q-btn icon="send" @click="handleSend" />
    </template>
  </div>
</template>
```

```

        </template>
    </q-input>
</div>
</template>

<script setup lang="ts">
import { ref } from 'vue';
import { useChat } from '@composables/useChat';

const { messages, sendMessage } = useChat();
const newMessage = ref('');

const handleSend = () => {
    if (newMessage.value.trim()) {
        sendMessage(newMessage.value, 'Usuário Anônimo');
        newMessage.value = '';
    }
};
</script>

<style scoped>
.chat-container {
    max-width: 600px;
    margin: 0 auto;
}
.message-list {
    height: 400px;
    overflow-y: auto;
}
</style>

```

Atividade Prática

Desafio: Adicione estas funcionalidades:

1. Nome do Usuário:

- Adicione um campo `q-input` para o usuário digitar seu nome antes de enviar mensagens.

2. "Digitando...":

- Publique um evento para `/app/typing` quando o usuário começar a digitar.
- Mostre uma mensagem temporária (ex: "João está digitando...").

Solução Sugerida:

typescript

```
// Em useChat.ts
const isTyping = ref(false);
const typingUsers = ref<string[]>([]);

// Adicione no onConnect:
stompClient.subscribe('/topic/typing', (message) => {
  typingUsers.value = JSON.parse(message.body);
});

// Novo método:
const sendTypingEvent = (sender: string) => {
  stompClient.publish({
    destination: '/app/typing',
    body: JSON.stringify(sender),
  });
};
```

vue

```
<!-- No template -->
<q-input
  @input="() => sendTypingEvent(username)"
/>
<div v-if="typingUsers.length">
  {{ typingUsers.join(', ') }} está digitando...
</div>
```

Recursos Adicionais

- Documentação Quasar: <https://quasar.dev/>
- STOMP.js: <https://stomp-js.github.io/>

Próxima Aula: Dashboard Analítico em Tempo Real com Charts.js! 📊

